

自进化编码智能体

Hao Zhou¹ Haichuan Hu¹ Ye Shang² Qianjun Zhang¹

¹Nanjing University of Science and Technology

²Nanjing University

125106010779@njjust.edu.cn

huhaichuan2024@gmail.com, yeshang@smail.nju.edu.cn, qianjunzhang@njjust.edu.cn

摘要

大语言模型正日益作为编码智能体嵌入软件工作流程中，它们能够检查代码仓库、调用工具、执行测试、调试故障并生成补丁。然而，现有的大多数智能体在部署后基本保持静态，尽管软件开发是一个动态且反馈丰富的过程，其中代码仓库不断演化、依赖关系发生变化、测试失败，且修复尝试会留下可复用的经验。这种矛盾催生了大量关于自进化编码智能体的研究，其中智能体通过从先前的编码交互中更新其框架、记忆、技能、工具、模型或协作结构来改进其未来行为。在本综述中，我们对这一新兴领域进行了系统综合。我们首先定义了自进化编码智能体，并将其与传统编码智能体和通用自进化智能体区分开来。随后，我们提出了一种以对象为中心的分类法，用以刻画这些系统中进化的内容，并辅以两个正交视角：进化发生的时间以及驱动进化的软件特定证据。纵观现有文献，我们发现可执行反馈、代码仓库级上下文和编码轨迹赋予了软件工程作为智能体自进化自然领域的独特地位，但同时也带来了反馈可靠性、基准过拟合、安全性、可维护性、成本和泛化性方面的新挑战。通过围绕这些维度组织现有工作，本综述旨在厘清自进化编码智能体的概念边界，并为设计更具适应性、可靠性和软件感知能力的智能系统奠定基础。我们收集的论文可在 <https://github.com/zhouhao1024/Awesome-Self-Evolving-Coding-Agents> 找到。

1 引言

大语言模型（Large Language Model）已迅速改变软件工程中自动化的角色。早期的代码助手主要专注于代码补全或函数级生成，但近期的编码智能体日益作为嵌入现实开发工作流的交互式系统运行 [Yang 等人, 2024; Wang 等人, 2024b]。它们能够解释自然语言需求、检查仓库结构、编辑多个文件、调用命令行工具、运行测试、诊断失败、生成补丁，并辅助代码审查与调试。这些能力尤为重要，因为软件工程任务很少是孤立的文本生成问题：它们是长周期、工具密集型的，并且与项目特定的代码库、依赖项、构建系统、测试套件及持续集成流水线紧密耦合 [Jimenez 等人, 2023]。因此，编码智能体正成为语言模型与真实软件工作流程之间的重要接口。

然而，编码智能体应用范围的扩大也暴露出静态智能体设计的局限性。在许多系统中，基础模型、提示、工具接口、记忆机制和控制流程在部署后基本固定。这一假设在现实的软件工程环境中难以维持：代码库持续演进，应用程序编程接口（API）和依赖关系不断变化，项目约定各不相同

跨仓库的缺陷修复通常需要反复进行定位、补丁生成、执行和修订的循环。同时，软件工程提供了丰富的可执行反馈，包括单元测试、编译器错误、运行时迹、静态检查警告、持续集成结果以及人工代码审查。如果编码智能体无法从这些反馈中积累经验，它们可能会在不同任务中重复类似的错误，并且无法适应项目特定的上下文。这些观察结果催生了自进化编码智能体：能够根据先前的编码尝试和软件特定反馈来更新其框架、记忆、技能、工具、模型或工作流与拓扑结构的智能体 [Robeyns 等人, 2025; Zhang 等人, 2025b; Xia 等人, 2025; Xiao 等人, 2026]。从这个意义上说，软件工程是研究自进化智能体的天然领域。

尽管近期工作已将自进化智能体 (Self-Evolving Agent) 作为一种通用范式进行探索 [高 (Gao) 等人, 2026, 方 (Fang) 等人, 2025]，但现有讨论大多聚焦于在广泛任务环境中提升智能体，而非分析软件工程的独特需求。这使得自进化编码智能体作为一个研究问题尚未得到充分刻画。与通用进化智能体相比，进化编码智能体在以代码仓库为中心的环境中运行，与编译器、测试框架、命令行界面、依赖管理器和持续集成系统交互，并从可执行工件 (Artifact) 中获得具体反馈。因此，其进化不仅与通用任务解决能力相关，还与软件特定的关注点紧密相连，例如代码仓库理解、迭代调试、代码正确性和可维护性。相应地，对其评估必须超越通用任务完成度，并考虑软件特定的标准，包括功能正确性、代码质量、安全性、效率、可维护性以及对误导性或不完整反馈的鲁棒性。

由于该领域仍处于快速兴起阶段，其概念边界尚不固定，本文将本综述定位为一种引导性综合，而非对已完全确立范式的回顾。我们不强行划定严格边界，而是旨在将编码智能体进化的异构机制组织成一个连贯的框架。为使问题具体化，本综述围绕三个研究问题展开：

- 研究问题一：编码智能体的哪些组件发生进化，通过何种机制实现进化？
- 研究问题二：编码智能体的进化何时发生，哪些软件特定的证据驱动这一过程？
- 研究问题三：如何从软件工程性能、可靠性和超越进化场景的泛化能力方面评估自进化编码智能体？

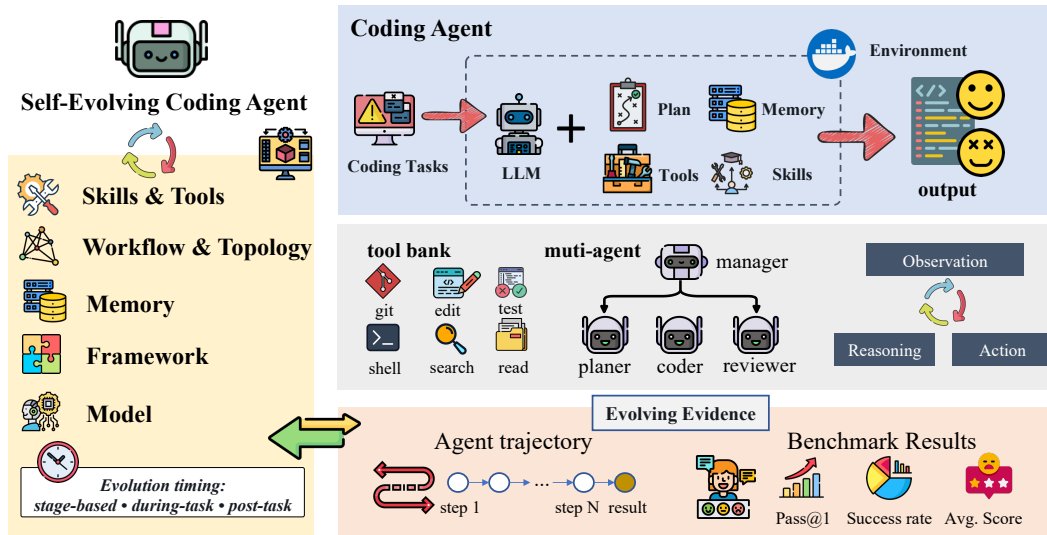


图 1：自进化编码智能体概览。

为回答这些问题，我们对自进化编码智能体进行了分层分析。首先厘清编码智能体、自进化智能体和自进化编码智能体之间的概念边界。然后围绕进化对象——智能体的哪个部分——组织现有工作。

当它从编码经验中学习时发生变化。这种以对象为中心的分类法使我们能够在统一的视角下比较那些演化智能体框架、记忆、技能与工具、模型侧组件或工作流与拓扑结构的系统，而不是将它们视为孤立的技术。我们进一步用两个正交维度来补充这一分类法：演化发生的时间以及驱动演化的软件特定证据。最后，我们讨论应如何评估此类智能体，超越一次性任务成功，转向正确性、可维护性、鲁棒性、成本、安全性以及超越演化发生环境的泛化能力。

通过这种分解，本综述为分析、比较和设计自演化编码智能体提供了一个结构化框架。我们的主要贡献如下：

- 我们阐明了自演化编码智能体的概念，并将其与代码生成模型、传统编码智能体以及通用自演化智能体区分开来。
- 我们提出了一种以对象为中心的分类法，根据智能体中演化的内容来组织现有工作，包括智能体框架、记忆、技能与工具、模型以及工作流或拓扑结构。
- 我们分析了塑造智能体演化的时间模式和软件特定证据，涵盖任务时、任务后和分阶段演化，以及结果证据、环境反馈和轨迹衍生证据。
- 我们综合了自演化编码智能体的评估实践和开放挑战，包括正确性、可维护性、鲁棒性、成本、安全性、基准设计以及超越演化环境的泛化能力。

2 背景与定义

在考察编码智能体如何演化之前，有必要澄清所研究的系统类型。编码智能体不仅仅是代码生成模型；它们处于软件工程环境中，语言模型与代码仓库、工具、测试和人类开发者进行交互。类似地，自演化不是简单的重复提示或一次性优化，而是一个由反馈驱动的过程，智能体通过该过程随时间改变其行为或内部组件。本节通过定义编码智能体、自演化智能体和自演化编码智能体，并为后续分类法和评估讨论指定范围，为本综述奠定概念基础。

2.1 编码智能体

编码智能体代表了从将代码生成视为孤立的文本生产，向将软件工程视为工具中介、环境接地的行动的范式转变。传统代码模型主要根据自然语言规范或局部上下文生成代码，而编码智能体则在开发环境中运行：它们检查代码仓库、调用工具、编辑文件、执行命令、观察反馈并迭代修改其解决方案。早期系统如 ChatDev 和 MetaGPT 将软件开发构建为角色专业化智能体之间的协作 [钱等人, 2023, 洪等人, 2023]。AgentCoder 通过协调程序员、测试设计者和测试执行者智能体，在代码生成中进一步实例化了这一思想 [黄等人, 2023]。更近期的系统如 SWE-agent 和 OpenHands 通过将智能体连接到终端、文件系统、代码仓库和执行环境，使这一范式更接近现实的软件工程 [杨等人, 2024, 王等人, 2024b]。

从系统视角来看，编码智能体将语言模型与上下文、工具、控制逻辑和验证机制相结合。模型提供推理和代码生成能力；控制器分解任务并选择动作；上下文机制维护仓库状态和任务历史；工具暴露搜索、编辑、测试、调试和依赖管理等操作；验证机制检查生成的更改是否满足可执行约束。现有系统以不同方式实例化这一设计空间。AutoDev 强调面向人工智能驱动开发的自主任务管理 [图法诺等人, 2024 年]。AutoCodeRover 和 RepairAgent 专注于仓库级程序修复和改进 [张等人, 2024 年 d, 布泽尼亚等人, 2024 年]。MASAI、CodeR 和 SpecRover 探索了针对软件工程任务的模块化、多智能体、任务图或意图感知设计 [阿罗拉等人, 2024 年, 陈等人, 2024 年, 阮等人, 2024 年]。Agentless 进一步表明，简化的定位、规划和修复阶段也可以高度

有效，这表明编码智能体的设计应通过其交互循环的质量来评估，而不仅仅是架构复杂度 [夏等人，2024]。

从 workflow 视角来看，编码智能体通过与软件工件的迭代交互来完成任务。典型的循环包括理解问题或请求、探索仓库、定位相关代码、编辑文件、运行测试、诊断失败并修订补丁。基于搜索的方法如 SWE-Search 强调了仓库级修复中探索和细化的重要性 [安东尼阿德斯等人，2024 年]。终端原生智能体进一步强调脚手架、测试装置设计和上下文工程作为可靠开发环境交互的关键因素 [布伊，2026 年]。其他工作研究可执行动作空间和测试执行作为智能体行为的核心部分 [王等人，2024 年 a，布泽尼亚和普拉德尔，2024 年]。

因此，编码智能体并非由单一基准或任务类型所定义，而是由其将语言模型用作软件工程 workflow 中决策组件的能力所界定。它们已被研究用于仓库级代码生成、代码审查、交互式调试以及仓库理解 [Zhang 等人，2024b；Garg 与 Huang，2026；Ma 等人，2026]。诸如 SWE-bench 和 SWE-Bench Pro 等基准阐明了此类智能体必须运行的环境：长时程、工具密集、反馈丰富，并与项目特定仓库紧密耦合 [Jimenez 等人，2023；Deng 等人，2025]。这些特性使编码智能体成为研究软件工程中自我演化的天然基础。

2.2 自我进化智能体

自我演化智能体通过将改进的着力点从外部工程更新转向智能体系统内部基于反馈的适应，从而扩展了传统的基于大语言模型的智能体。在这一视角下，自我演化并不仅仅意味着使用不同提示重新运行智能体，也不要求每次改进都更新模型参数。相反，它指的是智能体基于自身执行轨迹、环境反馈和累积经验来修改其行为或内部组件的能力。近期综述围绕几个反复出现的问题刻画了这一新兴范式：智能体的哪一部分在演化、演化何时发生、以及何种信号引导适应过程 [高等，2026，方等，2025]。这一视角催生了大量方法，其中智能体从反思、批评、任务结果、交互迹或自生成数据中学习。例如，Reflexion 和 Self-Refine 利用语言反馈和自我批评来改进后续决策，而无需改变模型权重 [希恩等，2023，马丹等，2023]；而 ExpeL 和 AGENT KB 则将过去的轨迹转化为可跨任务检索的可复用经验 [赵等，2024，唐等，2025]。

因此，理解自我进化智能体的一个有效方法是考察被更新的智能体组件。一些方法进化条件未来行为的上下文或提示，如提示优化和自指提示进化 [费尔南多等人，2023 年，哈塔布等人，2023 年，尤克塞尔戈努尔等人，2024 年]。另一些方法进化记忆系统，这些系统随时间存储、抽象和检索经验，使智能体能够复用先前的成功与失败，而不是将每个任务视为独立的 [帕克等人，2023 年，奇卡拉等人，2025 年，张等人，2025 年 a]。第三类工作进化技能和工具：Voyager 通过开放式交互构建可执行技能库，而近期以工具和技能为中心的系统研究智能体如何创建、选择、精炼和评估可复用能力 [王等人，2023 年，夏等人，2026 年，林等人，2026 年]。更激进的自我进化形式在模型行为、策略、workflow 或架构层面运作，包括自生成训练数据、基于反馈的强化学习、对智能体设计的进化搜索以及多智能体协同进化 [周等人，2025 年，张等人，2024 年 c，袁等人，2024 年，翁等人，2026 年]。这些研究共同表明，自我进化最好被理解为一个谱系：从通过反思和记忆进行的轻量级适应，到修改工具、workflow、策略或智能体架构的更强形式。这一通用范式为自我进化编码智能体提供了概念基础，但与大多数通用智能体设置相比，软件工程引入了更具体的工件、反馈信号和正确性约束。

2.3 自进化编码智能体

自进化编码智能体出现在两个近期趋势的交汇处：编码智能体在现实软件 engineering workflow 中的部署，以及对能够从自身经验中适应的智能体日益增长的兴趣。然而，它们并非简单地配备

一个额外的学习模块，也不仅仅是应用于代码的通用自进化智能体。传统编码智能体的主要特征在于其能够在软件环境中行动：它可以检查代码仓库、编辑文件、调用工具、运行测试、调试失败并生成补丁。自进化编码智能体则更进一步，将这些交互转化为持续适应的来源。在本综述中，我们使用该术语指代一种智能体软件工程系统，该系统根据先前的编码尝试和软件特定的反馈来更新其行为或内部组件。此类更新可能影响智能体框架本身，例如通过修改并验证自身实现的自改进编码智能体 [罗贝恩斯等人，2025]，或通过维护并在不断演化的编码智能体变体之间进行选择 [张等人，2025b，匿名，2026，王等人，2025a]。更新也可能在软件任务求解过程中在线发生，此时智能体根据其当前正在执行的轨迹进行适应 [夏等人，2025]。其他系统则专注于将编码轨迹蒸馏为可复用的技能或技能注册表，以指导后续的开发任务 [肖等人，2026，李等人，2026，谭等人，2026]。

自进化编码智能体的独特性在于软件工程循环中进化发生的本质。与许多通用自进化智能体不同，后者的反馈可能来自文本批评、用户偏好或标量奖励，编码智能体在可执行工件上运行，这些工件提供具体且可重复的信号。单元测试、编译器诊断、运行时迹、静态分析警告、仓库历史、持续集成日志和代码审查都可以成为适应的证据。这些信号使智能体能够为未来任务积累问题解决经验 [陈等人，2026 年]，构建用于定位和项目理解的仓库记忆 [王等人，2026 年 a]，并在安全关键场景（如漏洞修复）中重用基于经验的修复知识 [胡等人，2026 年 a]。在策略层面，软件演化数据可以进一步为改进智能体在开放式软件任务上的推理和决策提供训练信号 [魏等人，2025 年]。同时，这种反馈丰富的环境也使演化更加微妙：测试可能不完整，日志可能模糊，基准信号可能被过拟合，通过本地检查的补丁仍可能损害可维护性或安全性。因此，自进化编码智能体应被视为软件工程系统来研究，其演化基于可执行反馈、仓库级上下文和代码质量约束。

表 1：自进化编码智能体的概念边界。

Concept	Core idea	Feedback source	What changes
Coding agents	Act in software engineering workflows	Tool outputs, tests, and user instructions	Task state and generated patches
General self-evolving agents	Adapt behavior across tasks or environments	Textual feedback, rewards, and task outcomes	Prompts, memory, tools, policies, or architectures
Self-evolving coding agents	Adapt software engineering behavior through coding experience	Executable feedback from repositories, tests, CI, logs, and reviews	Repository memory, coding skills, workflows, policies, or scaffolds

3 自进化编码智能体的分类

本节从进化过程中实际更新的对象出发，构建自进化编码智能体的分类体系。我们不将自进化视为单一技术，而是将其视为作用于编码智能体系统中不同工件的一系列适应过程。在软件工程场景中，这些工件通常位于基础语言模型之外：智能体可以修订自身框架、积累修复经验、构建仓库记忆、提炼可复用的编码技能、创建或改进工具、重组工作流程、调整多智能体协作，或更新其底层模型策略。基于所调研的文献，我们将现有工作分为五类：智能体框架自进化、经验与仓库记忆自进化、技能与工具自进化、模型自进化，以及工作流与拓扑自进化。这些类别并非互斥，因为单个系统可能同时进化多个工件。分类旨在识别适应的主要对象，并使不同的自进化机制具有可比性。

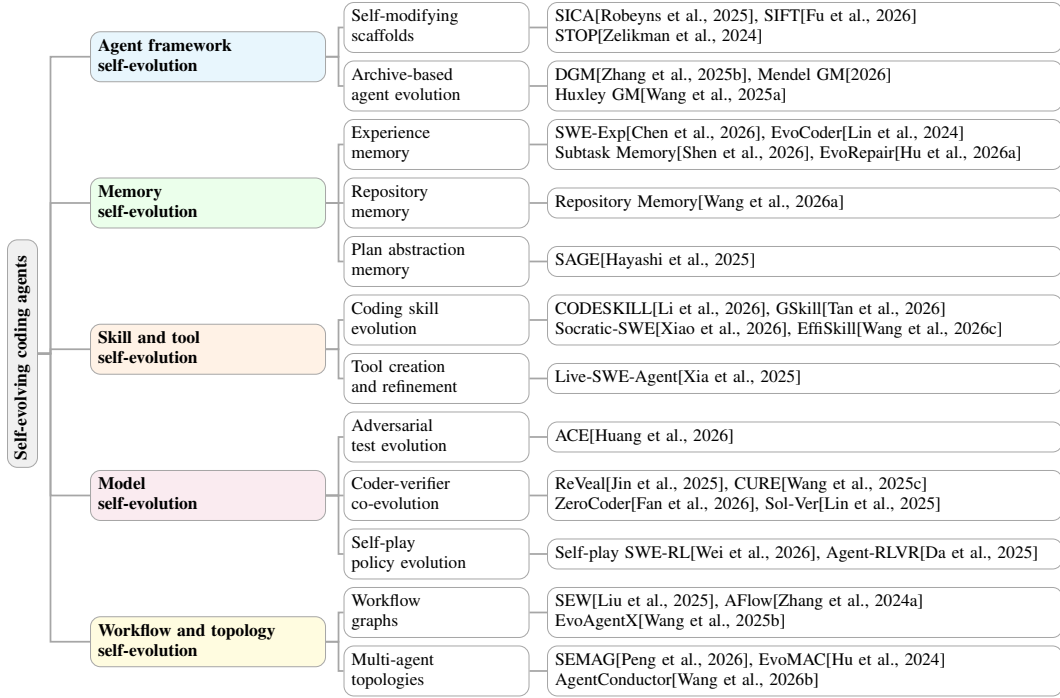


图 2: 按进化主要对象划分的自进化编码智能体分类体系。每个叶节点列出相应类别中的代表性系统。

表 2 对图 2 进行补充，将所调研的系统映射到本文使用的主要分析维度。我们纳入那些核心贡献通过编码特定反馈改变智能体组件或智能体行为的文献；仅包含基准数据集和静态编码智能体系统的文献将在后文作为评估背景讨论，而非作为自进化方法。

3.1 智能体框架自进化

智能体框架自演化将编码智能体本身视为可修改的软件工件。现代软件工程智能体通常围绕一个脚手架构建，该脚手架协调模型调用、仓库检查、文件编辑、shell 命令、测试执行、工具使用和控制流 [杨等人, 2024, 王等人, 2024b]。与通用自进化智能体不同，后者的演化通常作用于提示、记忆或高层策略，编码智能体暴露了一个更具体的适应目标：实现智能体本身的源代码和执行框架。这使得框架自演化在软件工程环境中尤为自然。智能体可以检查自己的实现，对其框架提出代码更改，执行修改后的版本，并通过软件特定的反馈（如测试结果、运行时故障、基准解决率或补丁有效性）评估结果。

现有工作遵循几种形式的框架自进化。最直接的形式是脚手架重写，即智能体修改其自身智能体系统的实现。一个自我改进的编码智能体通过为编码智能体配备基本软件工具并允许其编辑自身代码库、发现新的提示方案或工具，并在编码基准上验证所得智能体，展示了这一思想 [罗贝恩斯等人, 2025]。SIFT 遵循相同的脚手架级设置，但专注于使这种搜索更具样本效率：不是完全评估每个候选自我修改，而是使用大模型作为评审信号和轻量级树搜索来优先处理最有前景的补丁 [傅等人, 2026]。这个方向在概念上与 STOP 相关，后者研究递归自我改进的代码生成脚手架 [泽利克曼等人, 2024]。第二种形式是基于档案的框架进化，系统维护多个可执行的智能体变体并在自我修改上进行搜索。达尔文哥德尔机将这一过程构建为编码智能体变体的开放式进化，而后来的哥德尔机风格系统进一步研究如何

表 2: 代表性自进化编码智能体论文的分类。

Paper	Main object	Timing	Evidence	SWE task/domain
SICA [Robeyns et al., 2025]	Agent framework	Stage-wise	Outcome	Coding-agent development
SIFT [Fu et al., 2026]	Agent framework	Stage-wise	Outcome	Coding-agent development
STOP [Zelikman et al., 2024]	Agent framework	Stage-wise	Outcome	Code-generation agents
Darwin Gödel Machine [Zhang et al., 2025b]	Agent framework	Stage-wise	Outcome	Coding-agent development
Mendel Gödel Machine [Anonymous, 2026]	Agent framework	Stage-wise	Outcome	Coding-agent development
Huxley Gödel Machine [Wang et al., 2025a]	Agent framework	Stage-wise	Outcome	Coding-agent development
SWE-Exp [Chen et al., 2026]	Memory	Post-task	Trajectory-derived	Repository issue resolution
EvoCoder [Lin et al., 2024]	Memory	Post-task	Trajectory-derived	Defect reproduction
Subtask-Level Memory [Shen et al., 2026]	Memory	Post-task	Trajectory-derived	Repository-level issue resolution
EvoRepair [Hu et al., 2026a]	Memory	Post-task	Trajectory-derived	Vulnerability repair
Repository Memory [Wang et al., 2026a]	Memory	Post-task	Trajectory-derived	Code localization
SAGE [Hayashi et al., 2025]	Memory	Post-task	Trajectory-derived	Repository-level repair
CODESKILL [Li et al., 2026]	Skill/tool	Post-task	Trajectory-derived + environmental	Repository-level coding tasks
GSkill [Tan et al., 2026]	Skill/tool	Post-task	Trajectory-derived	Repository-level issue resolution
Socratic-SWE [Xiao et al., 2026]	Skill/tool	Post-task	Trajectory-derived	Repository-level issue resolution
EffiSkill [Wang et al., 2026c]	Skill/tool	Post-task	Environmental	Code efficiency optimization
Live-SWE-Agent [Xia et al., 2025]	Skill/tool	Task-time	Environmental	Repository issue resolution
Self-play SWE-RL [Wei et al., 2026]	Model	Stage-wise	Outcome + environmental	Bug generation and repair
Agent-RLVR [Da et al., 2025]	Model	Stage-wise	Environmental	Repository-level issue resolution
ReVeal [Jin et al., 2025]	Model	Stage-wise	Environmental	Code generation and verification
CURE [Wang et al., 2025c]	Model	Stage-wise	Environmental	Code generation and test generation
ZeroCoder [Fan et al., 2026]	Model	Stage-wise	Environmental	Code generation and test generation
Sol-Ver [Lin et al., 2025]	Model	Stage-wise	Environmental	Code generation and verification
ACE [Huang et al., 2026]	Model	Stage-wise	Environmental	Code generation and unit testing
SEW [Liu et al., 2025]	Workflow and topology	Task-time	Outcome + environmental	Code generation
AFlow [Zhang et al., 2024a]	Workflow and topology	Stage-wise	Outcome	Code generation and coding tasks
EvoAgentX [Wang et al., 2025b]	Workflow and topology	Stage-wise	Outcome	Code generation and coding tasks
SEMAG [Peng et al., 2026]	Workflow and topology	Task-time	Outcome + environmental	Multi-agent code generation
EvoMAC [Hu et al., 2024]	Workflow and topology	Task-time	Outcome + environmental	Multi-agent development
AgentConductor [Wang et al., 2026b]	Workflow and topology	Task-time	Environmental	Competition-level code generation

在谱系和任务之间选择、继承和评估自我修改 [Zhang 等人, 2025b, Anonymous, 2026, Wang 等人, 2025a]。

这一工作方向与更广泛的代码进化系统密切相关, 例如用于算法发现和优化的进化编码智能体 [诺维科夫等人, 2025 年, 阿松普桑等人, 2025 年, 胡等人, 2026 年 b]。然而, 软件工程智能体中的框架自进化与仓库级开发工作流的耦合更为紧密。进化的工件不仅是被优化的候选程序, 而且是产生未来软件工程动作的机制。框架变更成为可执行的智能体代码, 可以运行、调试并与先前的智能体版本进行比较。反馈循环也异常具体: 编译错误、单元测试、shell 输出、基准结果以及仓库级任务成功提供了关于框架修改是否改进智能体的直接证据。

与此同时, 与较轻量级的演化形式相比, 这一类别引发了更强的可靠性担忧。由于演化对象是生成未来动作的机制, 有害的框架修改可能会破坏智能体循环、降低工具使用能力、对基准反馈过拟合, 或利用评估框架中的弱点。因此, 框架自演化不仅需要性能驱动的搜索, 还需要仔细的验证、回滚和鲁棒性检查。

3.2 记忆自演化

记忆是编码智能体在单一任务之外积累软件工程经验的核心组件。在自演化编码智能体中，记忆自演化并不仅仅意味着存储交互历史。相反，它指的是对一个显式记忆组件的持续构建、精炼和复用，该组件记录软件特定的经验，例如问题解决轨迹、仓库历史、失败与成功的补丁、测试结果、编译器诊断信息、运行时日志、漏洞模式以及代码审查反馈。因此，被演化的对象是智能体的记忆机制：保留哪些信息、如何抽象、何时更新，以及如何检索以指导未来的软件工程动作。

这一视角尤为重要，因为软件工程任务很少是相互独立的。缺陷可能在相关模块间反复出现，相似的应用程序编程接口（API）可能以相似的方式失效，测试可能暴露出重复的失败模式，而仓库历史往往能揭示哪些组件倾向于一起变更。无记忆的编码智能体必须为每个新问题重新发现这些信息，而具备记忆进化能力的智能体则能将先前的编码尝试转化为可复用的知识。SWE-Exp 遵循这一方向，通过从先前的问题解决轨迹中构建经验库，包括成功和失败的修复尝试 [Chen 等人，2026]。由此产生的记忆使智能体在处理新的软件问题时能够复用先前的定位策略、修补决策和失败教训。EvoCoder 将类似的思想应用于问题代码复现，其中分层经验池将通用经验与仓库特定经验分离，并根据先前已解决的复现轨迹进行更新 [Lin 等人，2024]。结构对齐的子任务级记忆进一步在分析、定位、编辑和验证子任务的粒度上存储、检索和更新 SWE-智能体的经验，避免了对整个任务轨迹的粗粒度匹配 [Shen 等人，2026]。

记忆自我进化的第二种形式是以仓库为中心的记忆。仓库记忆不是将仓库视为静态的输入上下文，而是捕捉代码库随时间的演变方式。利用仓库记忆改进代码定位从历史提交、关联问题以及频繁修改代码区域的功能摘要中构建记忆，并利用该记忆支持未来的代码定位任务 [Wang 等人，2026a]。这种记忆类型是软件工程所特有的：它植根于代码库的时间结构、文件与模块的共同演化，以及问题报告与代码变更之间的历史关系。

记忆自进化也可以专门针对特定的软件工程领域。EvoRepair 研究漏洞修复，并引入了一种基于经验的自进化框架，该框架在漏洞内部积累修复经验，并在不同漏洞之间复用经验 [胡等人，2026a]。在这种设定下，记忆由补丁尝试、修复结果和漏洞特定的反馈所塑造，使其成为一个领域感知的知识库，而非过去交互的通用记录。这种记忆帮助智能体识别反复出现的漏洞模式，避免无效的修复动作，并为未来的安全任务检索相关的修复经验。

SAGE 提供了一种互补的轨迹衍生记忆形式：它将初始的软件工程智能体执行轨迹抽象为一个简洁的计划，并将该计划作为后续执行的上下文指导进行复用 [Hayashi 等人，2025 年]。

这些工作不同于 ExpeL 和 AGENT KB 等通用经验记忆系统，后者展示了跨任务存储和复用智能体经验的更广泛价值 [赵等人，2024 年，唐等人，2025 年]。然而，在自进化编码智能体中，记忆与可执行和仓库级别的证据结合得更为紧密。测试决定了记忆中的补丁策略是否真正正确，编译器和运行时错误揭示了具体的失败模式，提交历史提供了关于软件项目如何演变的长期信号。因此，记忆自进化将软件工程反馈转化为未来编码行为的内在、可复用的基础。

关键挑战不仅在于如何存储更多经验，更在于如何有选择地进化记忆。噪声日志、误导性测试、脆弱补丁和仓库特定约定都可能产生记忆，若不加批判地检索，反而会损害未来性能。因此，有效的记忆自进化需要过滤、抽象、检索和验证机制，使智能体能够从先前的软件经验中获益，同时避免对过往仓库、基准或偶然反馈产生过拟合。

3.3 技能与工具自进化

技能与工具的自进化关注编码智能体如何将软件工程经验转化为可复用的操作能力。记忆记录的是先前任务中发生了什么，而技能与工具编码的是当类似情境再次出现时智能体应如何行动。在软件工程中，这类能力尤为重要，因为有效的任务求解往往依赖于反复出现的流程：检查仓库结构、定位故障、选择相关测试、解读编译器或运行时错误、编辑补丁，以及通过执行验证变更。因此，这一类别中进化的对象并非代码库本身，而是智能体的程序性知识与工具使用能力：有哪些可用技能、何时应调用它们、如何更新它们，以及它们如何与特定软件工具（如外壳、测试运行器、代码搜索工具、静态分析器和补丁编辑器）交互。

一个具有代表性的方向是将编码轨迹提炼为可复用的程序性技能。CODESKILL 观察到，软件工程智能体在与代码仓库和终端环境交互时会产生丰富的轨迹，但原始轨迹过长且过于任务特定，无法直接复用 [李等人, 2026 年]。因此，它学习从编码智能体的轨迹中提取、演化并维护一个技能库。这些技能可以在不同粒度上运作：任务级技能捕获高层程序，例如如何检查代码仓库或验证修复；而事件驱动技能捕获对重复出现的执行事件（如命令失败、测试输出模式或反复出现的错误消息）的局部响应。重要的是，CODESKILL 将技能管理本身视为一个可学习的策略，同时利用基于评分标准的技能质量反馈和来自编码任务的可验证下游执行反馈。这使得技能演化不仅仅是一个摘要过程：智能体学习哪些程序性知识对未来的软件工程行为是有用的。

另一种技能自演化的形式专注于特定代码仓库的技能。面向编码智能体的自动技能学习引入了 gskill，这是一个为目标代码仓库学习简洁技能文档的流水线 [Tan 等人, 2026 年]。这些技能描述了代码仓库架构、编码规范、测试流程、常见陷阱以及典型的修改模式。关键的洞见是，编码智能体在不熟悉的代码仓库上的许多失败并不仅仅源于推理能力弱，而是由于缺少项目知识：智能体不知道代码仓库是如何组织的、测试应该如何运行，或者补丁必须遵循哪些规范。gskill 通过使用 SWE-smith 生成可验证的软件工程任务，然后通过一个演化优化循环迭代地精炼技能文档来解决这个问题。候选技能通过在隔离的代码仓库环境中运行智能体并检查其补丁是否通过测试来进行评估。从这个意义上说，特定代码仓库的技能充当了编码智能体自动学习的入门文档，并且它们的演化建立在可执行的软件反馈之上。

轨迹衍生的技能为闭环自进化提供了进一步的一步。苏格拉底式软件工程（Socratic-SWE）重用历史求解轨迹作为训练信号的来源，而不是在奖励计算后将其丢弃 [肖等人, 2026 年]。其轨迹包含软件特定的动作，如代码搜索、文件编辑、命令执行和测试运行。从这些轨迹中，系统提炼出一个智能体技能注册表（Agent Skill Registry），该注册表总结了反复出现的失败模式和有效的修复模式。这些技能随后指导在真实仓库中生成有针对性的修复任务，这些任务通过基于执行的验证进行过滤，并用于训练求解器。更新后的求解器产生新的轨迹，从而支持下一轮技能提炼。因此，苏格拉底式软件工程位于技能自进化和策略级自进化的交叉点：其显式的进化表示是智能体技能注册表，而该注册表塑造了未来的任务课程，并间接驱动求解器的改进。

技能自进化也可以针对特定的软件质量维度。高效技能（EffiSkill）通过从慢到快的程序对中挖掘可重用的优化技能，并将其组织成一个便携式工具箱，用于无需执行的诊断、技能检索、计划组合和候选生成，从而研究代码效率优化 [Wang 等人, 2026c]。尽管其技能库是离线构建的，而不是通过完全闭环的智能体循环构建的，但它对本分类法很有用，因为它展示了如何将反复出现的代码转换抽象为可供智能体在未来优化任务中使用的技能。

工具自进化与此密切相关，但在软件工程特定系统中目前发展较少。编码智能体已经严重依赖工具，包括 shell 命令、仓库搜索、依赖管理器、代码检查器、测试框架、调试器和补丁应用工具。然而，大多数现有的以软件工程为重点的工作提升的是智能体使用这些工具的技能，而不是使智能体能够自主创建或维护新工具。Live-SWE-Agent 提供了这一方向的一个软件工程特定示例：从一个仅含 bash 的最小脚手架开始，

智能体可以在解决仓库级问题时创建和修改自定义工具，如编辑器、代码搜索实用程序和领域特定的分析器 [夏等人，2025 年]。这些工具在问题解决循环中合成，并通过它们在仓库检查、编辑、执行和测试中的有用性进行评估。这为编码智能体指明了一条重要的未来路径：从学习如何使用固定工具转向为开发 workflow 创建、验证和维护项目特定的工具。

总体而言，技能和工具自进化将软件经验操作化。它将先前的轨迹、仓库约定、执行失败和修复模式转化为可复用的能力，以指导未来的行动。与强调存储和检索的记忆自进化相比，技能和工具自进化强调可操作性：智能体不仅应记住之前的尝试失败了，还应获得一个可复用的程序来避免类似的失败。核心挑战在于确保学习到的技能和工具既足够通用以跨任务迁移，又足够具体以在特定仓库和执行环境中发挥作用。

3.4 模型自进化

模型自进化是指改变编码智能体模型侧组件的适应过程，例如基础模型、适配器、智能体策略、奖励模型或验证器。这不同于记忆、技能或 workflow 自进化，后者中智能体可能改变其存储、检索或执行的内容，而底层模型保持不变。在软件工程中，模型自进化由异常具体的反馈所促成：测试、编译器诊断、执行迹、仓库状态和验证器判断可以被转化为训练信号，而不仅仅是指导单次修复尝试。关键边界在于，只有当软件特定的经验被反馈到控制后续智能体行为的模型侧组件时，普通的训练后过程才成为自进化。从这个意义上说，模型自进化与其说是关于选择监督微调或强化学习作为算法，不如说是关于编码轨迹、可执行结果或验证器判断是否成为智能体未来策略中的持久变化。

模型自我进化的最强形式出现在训练信号由智能体自身的软件交互生成时。自我博弈软件工程强化学习 (Self-play SWE-RL) 遵循这一方向，将缺陷生成、缺陷修复和可执行验证相结合：软件智能体在真实仓库中制造缺陷，尝试修复它们，并利用验证结果改进后续的求解器 [魏等人，2026 年]。智能体强化学习验证器 (Agent-RLVR) 提供了另一个例子，其中软件工程智能体首先生成轨迹，接收指导和环境奖励，然后利用引导性重试来更新智能体策略 [达等人，2025 年]。在这些系统中，可执行环境不仅仅是评估工具。它成为学习循环的一部分，将失败或成功的编码尝试转化为模型侧的更新。

模型自我进化也可以源于编码和验证能力的共同进化。ReVeal 交替进行代码生成和自我验证，利用解释器反馈和强化学习来同时提高生成器生成候选程序的能力和判断它们的能力 [金等人，2025]。CURE 和 ZeroCoder 通过同时训练编码器和单元测试器角色使这种关系更加明确：编码器通过面对越来越有信息量的测试而改进，而测试器通过暴露生成程序中的弱点而改进 [王等人，2025c，范等人，2026]。Sol-Ver 类似地将代码生成和测试生成构建为求解器-验证器自我博弈过程，表明编码智能体的验证侧可以是一个不断进化的模型组件，而不是固定的预言机 [林等人，2025]。ACE 通过对抗性单元测试生成和偏好优化进一步强化了选择压力，其中对手发现的失败案例成为改进求解器的证据 [黄等人，2026]。这些工作不仅仅是在评估中添加更多测试；它们将程序与测试之间的可执行分歧转化为对智能体未来行为的持续更新。

这一边界之所以重要，是因为许多近期的软件工程系统改进了模型，但并未完全构成自我进化的编码智能体。例如，软件工程强化学习 (SWE-RL) 表明，开放的软件演化数据和基于规则的奖励能够提升大语言模型在软件工程任务上的推理能力 [魏等人，2025]。然而，除非学习信号围绕智能体自身的演化尝试形成闭环，否则它更适合被理解为面向软件工程的模型优化。类似地，软件工程训练场 (SWE-Gym) 和可复现环境训练场 (R2E-Gym) 提供了可执行环境、轨迹和验证器信号，使得智能体策略改进成为可能，但它们的主要角色是基础设施而非自我进化本身 [潘等人，2025，贾因等人，2025]。软件工程奖励模型 (SWE-RM) 处于相关位置：它训练能够提供

免执行反馈的奖励模型，用于测试时扩展和强化学习，但该奖励模型是支持智能体改进的学习证据，而非完整的自我进化智能体 [Shum 等人，2025]。

因此，模型自我进化不仅取决于是否使用监督微调（SFT）或强化学习（RL），还取决于学习信号的来源以及它是否通过闭环软件反馈回路改变未来的智能体。在这方面，自生成任务尤其具有吸引力，但它们也暴露了一种失败模式：生成更多数据并不能保证进化。近期对自我博弈编码任务的分析表明，可持续的改进需要跨迭代的可学习信息增益；否则，该回路可能强化现有偏差或产生冗余任务，而无法改进下一代智能体 [刘等人，2026 年]。这使得模型自我进化既强大又脆弱。由于模型更新会影响跨任务的行为，不完整的测试、奖励黑客行为、合成数据工件或弱验证器可能教会智能体脆弱的习惯，这些习惯仅凭任务成功难以检测。

3.5 工作流与拓扑自我进化

工作流与拓扑自我进化将进化对象从单个智能体组件转移到智能体系统本身的组织结构。在编码任务中，这种组织并非表面的实现细节。编码智能体可能失败，并非因为其模型无法编写补丁，而是因为系统在编辑前定位了错误的文件、跳过了缺陷复现、过晚调用测试、将失败日志发送给错误的智能体，或迫使所有任务通过相同的僵化规划-编码-调试循环。由于编码任务从短函数综合到长周期软件开发各不相同，固定的协作协议变得越来越脆弱。因此，这一类的核心思想是让工作流、智能体角色和通信拓扑适应任务难度、执行反馈和验证需求。

这一视角拓展了早期依赖预定义协作结构的多智能体编码系统。ChatDev、MetaGPT 和 AgentCoder 表明，软件开发与代码生成能够从角色专业化、讨论、测试和审查中获益 [钱等人，2023，洪等人，2023，黄等人，2023]。然而，它们的角色和消息路径在很大程度上是人工设计的。近期自进化系统则将这些结构视为可变对象。例如，SEMAG 通过根据任务难度协调规划、编码、调试和讨论来调整多智能体代码生成工作流，同时允许模型选择随可用的编码骨干网络一同进化 [彭等人，2026]。EvoMAC 类似地将软件开发团队构建为一个多智能体协作网络，其智能体和连接可利用文本环境反馈、基于单元测试的验证以及文本反向传播进行更新 [胡等人，2024]。在此类系统中，测试结果和代码质量信号不仅用于评判最终程序，还为是否应修改生成该程序的协作模式提供了依据。

一种相关但更具结构性的视角将智能体过程表示为图。节点可对应规划、代码生成、重写、审查、测试、调试或选择，而边则指定信息流和执行顺序。SEW 表明，对于自动化代码生成，智能体提示词和工作流拓扑均可进化，因此不同的编码任务无需共享同一条手工构建的流水线 [刘等人，2025]。AFlow 通过蒙特卡洛树搜索和执行反馈在代码表示的工作流上进行搜索，从而推广了这一思想 [张等人，2024a]。EvoAgentX 进一步将此类工作流优化封装进一个更广泛的进化智能体框架中，联合优化提示词、工具和工作流拓扑，包括在代码生成任务上 [王等人，2025b]。这些系统强调，工作流进化并非简单地增加更多步骤，而是发现对于特定类别的编码问题，哪些验证、调试和优化路径值得激活。

拓扑演化关注智能体之间的通信结构。对于代码生成而言，有效的协作量取决于任务本身：简单任务可能因过度讨论而受损，而困难任务则需要规划者、编码者、调试者和审稿人之间更丰富的交互。AgentConductor 通过利用执行反馈为竞赛级代码生成生成任务自适应、密度感知的通信有向无环图，使这一权衡变得明确 [Wang 等人，2026b]。与 SEMAG 和 EvoMAC 一起，这一系列工作表明，协作应被视为一项软件工程决策：智能体不仅要决定编写什么代码，还要决定哪些角色应检查、测试、批评或修改该代码。

与记忆或技能演化相比，工作流和拓扑自演化改变了编码智能体更全局的层面。它决定了何时进行仓库搜索、调试是否与补丁生成分离、测试失败如何路由、哪个智能体审查补丁，以及值得为多少通信付出代价。这使得该类别功能强大但风险也高。演化出的工作流可能过拟合基准反馈，增加不必要的协调开销，或为了通过测试而忽视可维护性。因此，对于自演化编码智能体而言，挑战不仅在于发现更好的协作图，还在于确保这些图在现实的开发反馈下提升软件的正确性、效率和鲁棒性。

4 演化时间与证据

编码智能体的自我进化不仅取决于智能体被更新的部分，还取决于更新发生的时间背景以及更新所依赖的证据。在软件工程中，这两个方面紧密耦合。在单次调试尝试中观察到的编译器错误可能导致对当前补丁的立即修订，而跨问题的重复失败可能被整合到仓库记忆、可复用技能或用于后续模型更新的训练数据中。同样，测试失败、运行时迹、代码审查评论以及自我生成的修复任务，即使都表明当前智能体行为应当改变，它们所提供的监督类型也不相同。因此，自我进化的可靠性取决于智能体适应的速度、由此产生的变化持续的时间以及底层软件证据的可信度。本节将这两个维度——进化时间和进化证据——作为编码智能体如何将软件反馈转化为持续适应的互补视角进行考察。

4.1 进化时间

我们根据编码智能体更新其行为、组件或组织的时刻来对进化时间进行分类。在本综述中，我们区分了三种时间模式：任务时进化、任务后进化和阶段式进化。任务时进化发生在智能体仍在解决当前编码任务的过程中，例如当测试失败、编译器错误或工具失败导致对当前补丁、工具使用或工作流的立即更改时。任务后进化发生在任务或轨迹结束之后，此时智能体将结果抽象为记忆、技能、仓库知识或修复经验，以供后续任务使用。阶段式进化发生在积累了更大规模的证据之后，例如一批经过验证的轨迹、自我博弈任务、仓库交互或验证结果。这些时间模式在持久性和成本上有所不同：任务时进化快速且局部，任务后进化将个体结果转化为可复用的经验，而阶段式进化支持可能影响未来智能体版本的更广泛更新。

任务时进化。任务时演化（Task-time evolution）是指在单个编码任务内展开的自我演化。智能体不是等待任务完成，而是利用中间软件反馈来调整其正在进行的行为。这种设置在软件工程中尤为自然：失败的测试、编译器诊断、运行时迹、工具错误以及无效的仓库搜索，都能在最终补丁生成之前揭示当前策略的不足。由此产生的适应不仅可能影响候选代码，还可能影响所调用的工具、所遵循的调试路径或智能体之间的通信结构。

近期系统表明，任务时演化可以超越重试失败补丁的范畴。Live-SWE-Agent 在解决仓库级问题时创建并修改工具 [夏等人, 2025]。SEMAG 和 AgentConductor 通过演化工作流或通信拓扑，使多智能体代码生成适应任务难度 [彭等人, 2026, 王等人, 2026b]。SEW 和 EvoMAC 进一步利用代码生成反馈、单元测试验证和文本环境反馈来修改工作流结构或协作网络 [刘等人, 2025, 胡等人, 2024]。这些工作模糊了求解与演化之间的界限：暴露失败计划的同一条执行迹也可以指导智能体行为的即时重组。然而，此类适应通常局限于当前任务，当后续整合到记忆、技能、可复用工作流或模型级更新中时，其价值会更大。

任务后演化。任务后演化（Post-task evolution）发生在编码任务、问题解决尝试或开发轨迹结束之后。此时，智能体不再仅仅利用反馈来修复当前补丁；它可以将已完成的轨迹重新解释为未来行为的证据。这种

时间设置在软件工程中尤为重要，因为失败的测试、定位错误、补丁审查结果以及成功的修复迹，往往能揭示出从单个中间观察中无法看到的模式。一旦被抽象化，这些模式就能成为持久的经验、仓库知识、修复启发式方法或可复用的编码技能。

多个编码智能体系统通过将过去的软件工作转化为可复用的智能体状态，实现了这种形式的演化。一类工作将已完成的问题解决或修复轨迹视为可在后续任务中检索的经验，涵盖从问题解决记忆和仓库特定知识到漏洞修复经验 [陈等人，2026，王等人，2026a，胡等人，2026a]。近期的记忆系统进一步表明，任务后的证据并不局限于完整的情节：它可以被组织为层次化的复现经验、子任务对齐的迹或计划层面的抽象 [林等人，2024，沈等人，2026，林等人，2025]。另一类工作将轨迹蒸馏为可复用的编码技能，使得后续智能体不是由原始历史本身引导，而是由从先前开发尝试中学习到的抽象程序引导 [李等人，2026，谭等人，2026，肖等人，2026，王等人，2026c]。与任务时演化相比，任务后演化速度较慢但更具持久性：其价值在于将个体编码结果转化为可跨问题、仓库或未来智能体版本迁移的知识。

阶段式演化。阶段式演化描述了一种较慢但更持久的适应形式，其中智能体在积累了一批软件工程交互之后被更新。在这个时间尺度上，反馈不再仅用于修订当前补丁或存储单一教训。相反，软件轨迹、可执行结果、验证器判断或生成的修复任务被聚合为一个学习基底，塑造后续的智能体策略。这使得阶段式演化成为最接近跨智能体世代自我改进的时间形式，但它也需要更严格的边界：并非每个面向软件工程的后训练流程都是自我演化。反馈必须与智能体自身的尝试、生成的任务或交互结果相关联，而不仅仅是外部策划的训练数据集。

自我博弈软件工程强化学习（Self-play SWE-RL）为这种更强的形式提供了一个清晰的例子。它将缺陷生成、缺陷解决和可执行验证耦合在一起，使得软件智能体能够在真实仓库中创建越来越具有挑战性的缺陷，并利用由此产生的修复结果来改进后续的求解器 [魏等人，2026 年]。智能体强化学习与可验证奖励（Agent-RLVR）遵循一种相关的分阶段模式：智能体首先产生软件工程轨迹，接收指导和环境奖励，然后利用引导性重来来更新智能体策略 [达等人，2025 年]。这些系统不同于普通的模型后训练，因为学习信号是通过智能体与环境的交互、失败或成功的编码尝试以及可执行反馈产生的。相比之下，诸如软件工程强化学习（SWE-RL）这样的工作展示了软件演化数据对于训练软件工程推理模型的价值，但除非训练信号围绕智能体自身的演化行为形成闭环，否则它更适合被理解为面向软件工程的模型优化 [魏等人，2025 年]。

代码生成系统在生成与验证反复耦合时，呈现出一种相关的分阶段模式。在编码器-验证器与对抗性测试设置中，生成的程序、生成的测试以及执行结果成为后续模型行为得以强化的证据 [金等人，2025，王等人，2025c，范等人，2026，林等人，2025，黄等人，2026]。这些系统之所以与本文相关，并不仅仅因为它们使用了强化学习或偏好优化，而是因为下一个智能体是由先前的编码与验证尝试所塑造的。

这一区别也解释了为何分阶段演化依赖于可靠的训练环境与验证器。SWE-Gym 与 R2E-Gym 更适合被理解为这种演化形式的基础设施：它们提供了可执行的软件任务、轨迹以及验证器信号，从而使策略改进成为可能，但它们本身并非完整的自演化智能体 [潘等人，2025，贾因等人，2025]。类似地，诸如 SWE-RM 之类的奖励模型可以通过替代或补充昂贵的执行来支持分阶段更新，但它们充当的是学习到的证据，而非演化中的智能体本身 [舒姆等人，2025]。一个核心风险在于，更大规模的自生成或自动过滤数据并不必然产生更好的智能体。近期对自我博弈编码任务的分析表明，自演化需要在迭代过程中获得可学习的信息增益；否则，该循环可能只会生成更多冗余数据或强化既有偏差 [刘等人，2026]。

4.2 演化证据

自我演化的有效性不仅取决于智能体何时更新自身，还取决于更新所依据的证据。这一问题在软件工程中尤为突出，因为反馈是在不同的粒度级别上产生的。一些信号总结了尝试的结果，例如补丁是否解决了问题或通过了基准测试。另一些信号则在与软件环境交互的过程中产生，包括编译器诊断、测试日志、运行时错误和工具输出。还有一些信号只有在检查完整的编码轨迹后才能看到，其中成功和失败的动作共同揭示了可复用的修复策略、反复出现的错误或特定于代码库的实践。这些证据类型以不同的方式塑造自我演化：结果证据支持选择和强化，环境反馈指导持续适应，而轨迹衍生的证据则支持跨任务的经验、记忆和技能积累。

结果证据。结果证据概括了一次尝试性变更是否改善了可观测的软件工程性能。它可能表现为基准解决率、测试通过率、验证器得分、修复成功率，或结合准确率、成本与延迟的效用函数。此类证据虽粗略，但对自我进化至关重要，因为它提供了决定哪些智能体变体、工作流、技能或策略应被保留的选择压力。自我改进的编码智能体利用这类证据来验证其自身实现的变更：SICA 根据编码基准性能、成本与运行时间选择改进后的智能体版本 [罗贝恩斯等人，2025 年]，而达尔文哥德尔机及其后继者维护编码智能体变体的档案，并保留能改善经验编码性能的自我修改 [张等人，2025 年 b，匿名，2026 年，王等人，2025 年 a]。类似原理也出现在用于算法发现与优化的代码进化系统中，其中生成的程序或智能体修改根据基于执行的性能度量进行选择 [诺维科夫等人，2025 年，阿松普考等人，2025 年，胡等人，2026 年 b]。结果证据同样驱动工作流与拓扑结构的进化，其中代码生成性能、单元测试验证与任务结果决定应保留哪些协作结构 [彭等人，2026 年，刘等人，2025 年，胡等人，2024 年，王等人，2026 年 b]。在模型层面，可验证的软件任务结果可转化为策略更新或编码器-验证器训练信号 [魏等人，2026 年，达等人，2025 年，金等人，2025 年，黄等人，2026 年]。结果证据的优势在于候选更新之间的可比性；其弱点在于它往往只能识别哪个智能体更优，而无法解释改进为何发生。

环境反馈。环境反馈产生于智能体与软件环境的交互过程中。它包括命令输出、编译器诊断信息、运行时异常、失败的测试日志、调试器观察结果、依赖失败以及工具响应。这类证据比结果证据更具局部性：它不仅评判整个尝试，还揭示当前策略成功、停滞或失效的中间条件。Live-SWE-Agent 将这种反馈作为在线工具演化的直接触发因素，而 EvoMAC 则利用文本环境信号和单元测试验证，通过多智能体开发网络传播改进 [夏等人，2025，胡等人，2024]。同类证据也支持技能形成：反复出现的命令失败、测试输出模式、运行时行为和优化迹可以被抽象为可复用的编码技能，而不是在单次运行后丢弃 [李等人，2026，谭等人，2026，王等人，2026c]。在模型层面，环境奖励、沙箱执行、生成的测试和对抗性失败将策略或验证器的更新建立在可观察的软件行为之上 [达等人，2025，魏等人，2026，金等人，2025，王等人，2025c，范等人，2026，林等人，2025，黄等人，2026]。因此，环境反馈是任务内适应、工具创建和面向调试的演化的主要证据来源，尽管在支持更长期的自我改进之前，它通常需要被抽象化。

轨迹衍生证据。轨迹衍生证据来自智能体尝试的完整记录，而非单一评分或观察。编码轨迹包含仓库检查、故障定位、工具调用、文件编辑、测试执行、失败分支、恢复步骤和最终补丁提交。这类轨迹尤其有价值，因为它们不仅揭示尝试是否成功，还揭示其展开过程。在自我演化的编码智能体中，轨迹衍生证据最常用于构建记忆和技能。问题解决、定位、复现和修复轨迹可以被压缩为经验库或仓库知识，以指导未来任务 [Chen 等人，2026；Wang 等人，2026a；Lin 等人，2024；Hu 等人，2026a]。其他工作则根据轨迹的内部结构进行抽象，例如作为子任务级记忆或计划级摘要，

这使得存储的经验比原始日志更容易复用 [沈等人, 2026, 林等人, 2025]。基于技能的系统将同样的原理应用于程序性知识: 编码轨迹、修复尝试和优化迹被提炼为技能或技能注册表, 从而可以调节后续智能体 [李等人, 2026, 谭等人, 2026, 王等人, 2026c, 肖等人, 2026]。与结果证据相比, 轨迹衍生的证据更难处理; 与环境反馈相比, 它不够即时。其价值在于抽象: 它将具体的软件尝试转化为可复用的经验、记忆、技能和课程, 从而能够跨任务塑造未来的智能体行为。

5 基准与评估

评估在自进化编码智能体的研究中扮演双重角色。它不仅是衡量智能体性能的手段, 也是智能体进化的主要证据来源之一。基准结果、失败的测试、验证器判断或代价高昂的轨迹都可能作为信号, 用于决定是否应保留记忆项、是否应复用技能、是否应修订工作流, 或是否应更新模型侧组件。因此, 针对自进化编码智能体的评估必须超越一次性任务成功。它应捕捉正在解决的软件工程任务、执行期间和执行后可用的证据, 以及累积经验导致持续改进的程度。

5.1 评估任务与基准

仓库级问题解决。仓库级问题解决已成为自进化编码智能体的核心评估场景。与函数级代码生成不同, 这些任务要求智能体在真实的软件项目中运作: 它们必须理解问题描述、检查仓库结构、定位相关文件、编辑代码、运行测试, 并根据可执行反馈修改补丁。SWE-bench 通过收集真实的 GitHub 问题及其对应的拉取请求, 并配合基于执行的验证, 引入了这一场景 [Jimenez 等人, 2023 年]。其变体, 包括 SWE-bench Lite、SWE-bench Verified 和 SWE-Bench Pro, 进一步细化了仓库级评估的难度、验证质量和长程特性 [Jimenez 等人, 2023 年, Deng et al., 2025]。SWE-Gym 通过将软件工程任务转化为智能体和验证器的可执行训练与评估环境, 扩展了这一方向 [潘等人, 2025]。这些基准与自进化尤其相关, 因为仓库上下文、执行结果、失败尝试和补丁结果都可以成为未来适应的经验。

函数级和竞赛式编程。函数级和竞赛风格的编程基准仍然有用, 但它们扮演着不同的角色。人类评估 (HumanEval) 评估从文档字符串进行 Python 程序合成的功能正确性, 而 MBPP 则侧重于带有自然语言规范和测试的简短入门级编程任务 [Chen 等人, 2021, Austin 等人, 2021]。APPS 和 CodeContests 转向更困难的竞赛编程环境, 而 LiveCodeBench 则强调污染感知和持续更新的代码评估 [Hendrycks 等人, 2021, Li 等人, 2022, Jain 等人, 2024]。这些基准通常用于评估 SEMAG 和 SEW 等系统中的代码生成能力、算法推理和工作流优化 [Peng 等人, 2026, Liu 等人, 2025]。它们提供了受控的比较, 但并未完全捕捉到与仓库、依赖项、测试和 CI 系统进行长期交互的特征, 而这种交互正是现实软件工程的特点。因此, 它们最好被视为补充证据, 而不是仓库级评估的替代品。

5.2 评估度量

现有的评估通常从面向结果的度量开始, 例如通过率、解决率、解决率、修复率、基准分数和 Pass@k。这些度量是必要的, 因为它们表明最终输出是否满足基准的验证程序, 并且它们在仓库级问题解决、基于工作流的代码生成和竞赛风格编程中被广泛使用 [夏等人, 2025, 陈等人, 2026, 彭等人, 2026, 刘等人, 2025]。然而, 对于自我进化的编码代理来说, 最终的成功只是一个部分信号。更高的分数表明代理表现更好, 但它并不能解释改进是来自记忆、技能、工作流变化、验证器反馈还是模型侧的适应。

因此，更具信息量的评估应当揭示演化过程本身。一些系统追踪智能体修改是否在成本和时间的约束下提升了基准性能，如 SICA [罗贝恩斯等人，2025]；另一些系统则维护改进智能体变体的档案，如达尔文哥德尔机 [张等人，2025b]。基于技能和经验的系统评估累积的轨迹、习得的技能或仓库知识是否改进了未来任务 [李等人，2026，谭等人，2026，肖等人，2026，陈等人，2026]。可执行反馈进一步丰富了这些度量：单元测试结果、回归避免、验证器判断和奖励信号既可以作为评估标准，也可以作为演化的证据 [魏等人，2026，潘等人，2025，魏等人，2025]。

效率和泛化也很重要，因为自我演化通常需要额外的搜索、重复执行、检索、轨迹存储或模型更新。因此，一些工作报告了成本、运行时间、令牌使用量、步骤数或检索开销 [Robeyns 等人，2025；Chen 等人，2026；Wang 等人，2026a；Xia 等人，2025]。另一些工作测试演化出的能力是否迁移到保留的仓库、新基准、不同模型或不同编程语言 [Li 等人，2026；Xiao 等人，2026；Tan 等人，2026]。总体而言，当前的评估在衡量功能正确性和基准成功方面最强，但在评估长期可维护性、鲁棒性、安全性以及智能体是否从不完整或误导性的软件反馈中学习可靠行为方面较弱。

6 挑战与开放问题

自进化编码智能体带来的挑战超越了传统编码智能体，因为其行为会随时间变化。不可靠的测试结果、噪声轨迹、弱验证器或基准特定的捷径不仅会影响单个补丁，还可能被存储为记忆、提炼为技能、选为工作流，或用于更新模型。因此，关键挑战不仅在于自进化是否提升基准性能，还在于进化过程是否保持可靠、可复现，并符合软件工程约束。

可复现性、污染和基准过拟合。自我演化使可复现性变得困难，因为智能体可能在不同运行、任务、仓库、工具环境或模型版本之间发生变化。使用基准结果选择自我修改或智能体变体的系统对评估噪声和基准泄漏特别敏感 [罗贝恩斯等人，2025 年，张等人，2025 年 b]。这一担忧在代码评估中被放大，其中污染和基准特定适应已经是已知问题 [贾因等人，2024 年]。未来的评估必须区分真正的改进与记忆、重复的基准调整或对公开验证信号的过拟合。

反馈可靠性、安全性和工具依赖性。可执行反馈对于自进化编码智能体至关重要，但测试、编译器、持续集成日志、生成测试和奖励模型并不完美。因此，依赖单元测试验证、环境奖励或学习型验证器的系统可能会继承这些信号的偏差和盲点 [魏等人，2025，潘等人，2025]。当智能体修改工具、工作流或自身脚手架时，这种风险尤为突出，因为误导性的反馈信号可能塑造未来行为，而不仅仅是影响单次输出。因此，工具可靠性、沙箱保真度和安全检查是自进化问题的一部分，而不仅仅是实现细节。

长期记忆、技能与协调。记忆和技能机制使智能体能够复用软件工程经验，但它们也引入了质量控制问题。经验库、仓库记忆和技能库可能变得过时、冗余、过度依赖特定仓库，或被失败轨迹污染 [Chen 等人，2026；Li 等人，2026；Xiao 等人，2026]。多智能体和演化工作流系统进一步增加了协调挑战，因为演化的角色、通信模式或拓扑结构可能提升性能，但也可能增加成本、不稳定性 and 责任模糊性 [Liu 等人，2025]。

超越短期基准的评估。当前评估主要通过通过率、解决率或基准分数来衡量短期成功。然而，真实的软件工程还需要可维护性、安全性、可审查性、效率和长期可靠性。现有证据主要支持领域内或近领域泛化，例如跨编码基准、仓库或相关软件任务的迁移 [Li 等人，2026，Xiao 等人，2026]。进化所获得的

从软件工程反馈迁移到非编码领域在很大程度上仍未得到探索。因此，未来工作不仅应评估智能体在其演化环境中是否有所改进，还应评估演化后的行为在原始基准或代码库设定之外是否依然稳健。

7 结论

自进化编码智能体标志着从静态软件工程助手向能够通过与代码、代码仓库、工具、测试和人类反馈持续交互而改进的系统的转变。本文综述的文献表明，这一转变不应被理解为单一算法技术，而应视为基于可执行软件工件和代码仓库级上下文的更广泛的适应过程家族。这一场景的独特之处也正是其困难所在：软件工程为进化提供了具体反馈，但该反馈往往不完整、模糊、代价高昂或与短期基准挂钩。因此，未来工作的核心挑战不仅在于使编码智能体能够进化，更在于使其进化值得信赖。进展将需要能够验证反馈、修订过时记忆、审计所学技能、约束自我修改以及评估超越即时任务成功的长期软件质量的机制。若要使自进化编码智能体在真实软件工程 workflow 中具备适应性、可靠性和真正实用性，这些基础是必不可少的。

参考文献

- Anonymous. Mendel Gödel machine: Comparative evolution enables state-of-the-art self-improving coding agents, 2026. Manuscript under review.
- Antonis Antoniadis, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. SWE-Search: Enhancing software agents with monte carlo tree search and iterative refinement, 2024. URL <https://arxiv.org/abs/2410.20285>.
- Daman Arora, Atharv Sonwane, Nalin Wadhwa, Abhav Mehrotra, Saiteja Utpala, Ramakrishna Bairi, Aditya Kanade, and Nagarajan Natarajan. MASAI: Modular architecture for software-engineering AI agents, 2024. URL <https://arxiv.org/abs/2406.11638>.
- Henrique Assumpcao, Diego Ferreira, Leandro Campos, and Fabricio Murai. CodeEvolve: An open-source evolutionary framework for algorithmic discovery and optimization, 2025. URL <https://arxiv.org/abs/2510.14150>.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Islem Bouzenia and Michael Pradel. You name it, i run it: An LLM agent to execute tests of arbitrary projects, 2024. URL <https://arxiv.org/abs/2412.10133>.
- Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair, 2024. URL <https://arxiv.org/abs/2403.17134>.
- Nghi D. Q. Bui. Building AI coding agents for the terminal: Scaffolding, harness, context engineering, and lessons learned, 2026. URL <https://arxiv.org/abs/2603.05344>.
- Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. CodeR: Issue resolving with multi-agent and task graphs, 2024. URL <https://arxiv.org/abs/2406.01304>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders,

- Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution, 2026. URL <https://arxiv.org/abs/2507.23361>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- Jeff Da, Clinton Wang, Xiang Deng, Yuntao Ma, Nikhil Barhate, and Sean Hendryx. Agent-RLVR: Training software engineering agents via guidance and environment rewards, 2025. URL <https://arxiv.org/abs/2506.11425>.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?, 2025. URL <https://arxiv.org/abs/2509.16941>.
- Lishui Fan, Mouxiang Chen, Tingwei Zhu, Kui Liu, Xin Xia, Shanping Li, and Zhongxin Liu. ZeroCoder: Can LLMs improve code generation without ground-truth supervision?, 2026. URL <https://arxiv.org/abs/2604.07864>.
- Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems, 2025. URL <https://arxiv.org/abs/2508.07407>.
- Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution, 2023. URL <https://arxiv.org/abs/2309.16797>.
- Xinghong Fu, Aravindh Kulanthaivelu, and Yutaro Yamada. Self-improvement via fast tree-search. In *International Conference on Learning Representations*, 2026.
- Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence, 2026. URL <https://arxiv.org/abs/2507.21046>.
- Spandan Garg and Yufan Huang. Debug2Fix: Supercharging coding agents with interactive debugging capabilities, 2026. URL <https://arxiv.org/abs/2602.18571>.
- Hiroaki Hayashi, Bo Pang, Wenting Zhao, Ye Liu, Akash Gokul, Srijan Bansal, Caiming Xiong, Semih Yavuz, and Yingbo Zhou. Self-abstraction from grounded experience for plan-guided policy refinement, 2025. URL <https://arxiv.org/abs/2511.05931>.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS, 2021. URL <https://arxiv.org/abs/2105.09938>.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework, 2023. URL <https://arxiv.org/abs/2308.00352>.

- Haichuan Hu, Guoqing Xie, Qunjun Zhang, Jiawei Liu, Shengcheng Yu, Chunrong Fang, Zhenyu Chen, and Liang Xiao. EvoRepair: Enhancing vulnerability repair agents through experience-based self-evolution, 2026a. URL <https://arxiv.org/abs/2605.30105>.
- Tu Hu, Ronghao Chen, Shuo Zhang, Jianghao Yin, Mou Xiao Feng, Jingping Liu, Shaolei Zhang, Andy Wang, Wenqi Jiang, Yuqi Fang, Sen Hu, Huacan Wang, and Yi Xu. Controlled self-evolution for algorithmic code optimization, 2026b. URL <https://arxiv.org/abs/2601.07348>.
- Yue Hu, Yuzhu Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, Yuchen Hou, Shuo Tang, and Siheng Chen. Self-evolving multi-agent collaboration networks for software development, 2024. URL <https://arxiv.org/abs/2410.16946>.
- Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. AgentCoder: Multi-agent-based code generation with iterative testing and optimisation, 2023. URL <https://arxiv.org/abs/2312.13010>.
- Yixu Huang, Xinglei Yu, and Zhongyu Wei. ACE: Self-evolving LLM coding framework via adversarial unit test generation and preference optimization, 2026. URL <https://arxiv.org/abs/2605.16299>.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents, 2025. URL <https://arxiv.org/abs/2504.07164>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues?, 2023. URL <https://arxiv.org/abs/2310.06770>.
- Yiyang Jin, Kunzhao Xu, Hang Li, Xueting Han, Yanmin Zhou, Cheng Li, and Jing Bai. ReVeal: Self-evolving code agents via iterative generation-verification, 2025. URL <https://arxiv.org/abs/2506.11442>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines, 2023. URL <https://arxiv.org/abs/2310.03714>.
- Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. CODESKILL: Learning self-evolving skills for coding agents, 2026. URL <https://arxiv.org/abs/2605.25430>.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode, 2022. URL <https://arxiv.org/abs/2203.07814>.
- Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. MUSE-Autoskill: Self-evolving agents via skill creation, memory, management, and evaluation, 2026. URL <https://arxiv.org/abs/2605.27366>.
- Yalan Lin, Yingwei Ma, Rongyu Cao, Binhua Li, Fei Huang, Xiaodong Gu, and Yongbin Li. LLMs as continuous learners: Improving the reproduction of defective code in software issues, 2024. URL <https://arxiv.org/abs/2411.13941>.
- Zi Lin, Sheng Shen, Jingbo Shang, Jason Weston, and Yixin Nie. Learning to solve and verify: A self-play framework for code and test generation, 2025. URL <https://arxiv.org/abs/2502.14948>.

- Siwei Liu, Jinyuan Fang, Han Zhou, Yingxu Wang, and Zaiqiao Meng. SEW: Self-evolving agentic workflows for automated code generation, 2025. URL <https://arxiv.org/abs/2505.18646>.
- Wei Liu, Siya Qi, Yali Du, and Yulan He. Self-play only evolves when self-synthetic pipeline ensures learnable information gain, 2026. URL <https://arxiv.org/abs/2603.02218>.
- Dongjian Ma, Silin Chen, Yufei Yang, Yulin Shi, Yanfu Yan, and Xiaodong Gu. LLM agents can see code repositories, 2026. URL <https://arxiv.org/abs/2606.14061>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023. URL <https://arxiv.org/abs/2303.17651>.
- Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym, 2025. URL <https://arxiv.org/abs/2412.21139>.
- Yulin Peng, Haowen Hou, Xinxin Zhu, Ying Tiffany He, and F. Richard Yu. SEMAG: Self-evolutionary multi-agent code generation, 2026. URL <https://arxiv.org/abs/2603.15707>.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development, 2023. URL <https://arxiv.org/abs/2307.07924>.
- Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent, 2025. URL <https://arxiv.org/abs/2504.15228>.
- Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. SpecRover: Code intent extraction via LLMs, 2024. URL <https://arxiv.org/abs/2408.02232>.
- Kangning Shen, Jingyuan Zhang, Chenxi Sun, Wencong Zeng, and Yang Yue. Structurally aligned subtask-level memory for software engineering agents, 2026. URL <https://arxiv.org/abs/2602.21611>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- KaShun Shum, Binyuan Hui, Jiawei Chen, Lei Zhang, X. W., Jiaxi Yang, Yuzhen Huang, Junyang Lin, and Junxian He. SWE-RM: Execution-free feedback for software engineering agents, 2025. URL <https://arxiv.org/abs/2512.21919>.
- Shangyin Tan, Lakshya A. Agrawal, Rohit Sandadi, Dan Klein, Koushik Sen, Alexandros G. Dimakis, and Matei Zaharia. Automatically learning skills for coding agents. In *Proceedings of the ACM Conference on AI and Agentic Systems*, 2026. doi: 10.1145/3786335.3813196. URL <https://doi.org/10.1145/3786335.3813196>.
- Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, Ge Zhang, Jiaheng Liu, Xingyao Wang, Sirui Hong, Chenglin Wu, Hao Cheng, Chi Wang, and Wangchunshu Zhou. AGENT KB: Leveraging cross-domain experience for agentic problem solving, 2025. URL <https://arxiv.org/abs/2507.06229>.

- Michele Tufano, Anisha Agarwal, Jinu Jang, Roshanak Zilouchian Moghaddam, and Neel Sundaresan. AutoDev: Automated AI-driven development, 2024. URL <https://arxiv.org/abs/2403.08299>.
- Boshi Wang, Weijian Xu, Yunsheng Li, Mei Gao, Yujia Xie, Huan Sun, and Dongdong Chen. Improving code localization with repository memory, 2026a. URL <https://arxiv.org/abs/2510.01003>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL <https://arxiv.org/abs/2305.16291>.
- Siyu Wang, Ruotian Lu, Zhihao Yang, Yuchao Wang, Yanzhou Zhang, Lei Xu, Qimin Xu, Guojun Yin, Cailian Chen, and Xinping Guan. AgentConductor: Topology evolution for multi-agent competition-level code generation, 2026b. URL <https://arxiv.org/abs/2602.17100>.
- Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley Gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine, 2025a. URL <https://arxiv.org/abs/2510.21614>.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents, 2024a. URL <https://arxiv.org/abs/2402.01030>.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents, 2024b. URL <https://arxiv.org/abs/2407.16741>.
- Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. EvoAgentX: An automated framework for evolving agentic workflows, 2025b. URL <https://arxiv.org/abs/2507.03616>.
- Yinjie Wang, Ling Yang, Ye Tian, Ke Shen, and Mengdi Wang. CURE: Co-evolving LLM coder and unit tester via reinforcement learning, 2025c. URL <https://arxiv.org/abs/2506.03136>.
- Zimu Wang, Yuling Shi, Mengfan Li, Zijun Liu, Jie M. Zhang, Chengcheng Wan, and Xiaodong Gu. EffiSkill: Agent skill based automated code efficiency optimization, 2026c. URL <https://arxiv.org/abs/2603.27850>.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution, 2025. URL <https://arxiv.org/abs/2502.18449>.
- Yuxiang Wei, Zhiqing Sun, Emily McMilin, Jonas Gehring, David Zhang, Gabriel Synnaeve, Daniel Fried, Lingming Zhang, and Sida Wang. Toward training superintelligent software agents through self-play SWE-RL, 2026. URL <https://arxiv.org/abs/2512.18552>.
- Zhaotian Weng, Antonis Antoniadis, Deepak Nathani, Zhen Zhang, Xiao Pu, and Xin Eric Wang. Group-evolving agents: Open-ended self-improvement via experience sharing, 2026. URL <https://arxiv.org/abs/2602.04837>.
- Bowei Xia, Mengkang Hu, Shijian Wang, Jiarui Jin, Wenxiang Jiao, Yuan Lu, Kexin Li, and Ping Luo. Tool-genesis: A task-driven tool creation benchmark for self-evolving language agent, 2026. URL <https://arxiv.org/abs/2603.05578>.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents, 2024. URL <https://arxiv.org/abs/2407.01489>.
- Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-SWE-agent: Can software engineering agents self-evolve on the fly?, 2025. URL <https://arxiv.org/abs/2511.13646>.

- Chuan Xiao, Zhengbo Jiao, Shaobo Wang, Wei Wang, Bing Zhao, Hu Wei, Linfeng Zhang, and Lin Qu. Socratic-SWE: Self-evolving coding agents via trace-derived agent skills, 2026. URL <https://arxiv.org/abs/2606.07412>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. EvoAgent: Towards automatic multi-agent generation via evolutionary algorithms, 2024. URL <https://arxiv.org/abs/2406.14228>.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic “differentiation” via text, 2024. URL <https://arxiv.org/abs/2406.07496>.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling*, 2024. URL <https://arxiv.org/abs/2310.02304>.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. MemEvolve: Meta-evolution of agent memory systems, 2025a. URL <https://arxiv.org/abs/2512.18746>.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents, 2025b. URL <https://arxiv.org/abs/2505.22954>.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation, 2024a. URL <https://arxiv.org/abs/2410.10762>.
- Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. CodeAgent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges, 2024b. URL <https://arxiv.org/abs/2401.07339>.
- Wenqi Zhang, Ke Tang, Hai Wu, Mengna Wang, Yongliang Shen, Guiyang Hou, Zeqi Tan, Peng Li, Yueting Zhuang, and Weiming Lu. Agent-Pro: Learning to evolve via policy-level reflection and optimization, 2024c. URL <https://arxiv.org/abs/2402.17574>.
- Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement, 2024d. URL <https://arxiv.org/abs/2404.05427>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144>.
- Yifei Zhou, Sergey Levine, Jason Weston, Xian Li, and Sainbayar Sukhbaatar. Self-challenging language model agents, 2025. URL <https://arxiv.org/abs/2506.01716>.